

第9章 异常处理与并发

异常体系 · GIL 真相 · threading 与锁 · queue · asyncio 入门

本章目标:系统讲透两件事:Python 的异常处理体系(try/except/else/finally、raise from、自定义异常),以及并发编程(线程、GIL、锁与死锁、queue、多进程、asyncio 入门)。学完你能写出错误可查、并发安全的生产代码,并理解 Python 并发模型的边界与选型。

直觉起点:C 用返回码与 errno 报告错误,漏查就静默;Python 用异常,不处理就直接抛穿。并发侧,C 的 pthread 是"真并行";Python 的线程受 GIL 约束是"并发不并行"——理解 GIL 的成因,才能正确选型线程、进程与协程。

前置要求:第4章函数与作用域、第6章类与继承(异常也是类)、第8章网络(超时与重试已用到异常)。

本章路线:第一阶段(1.5 小时)读 §9.1–§9.2,掌握异常体系;第二阶段(2 小时)读 §9.3–§9.6,理解 GIL 并写出线程安全代码;第三阶段(2 小时)读 §9.7–§9.8,掌握多进程与 asyncio,完成章末实验。

本章知识导图

- └─ 9.1 异常体系 —— try/except/else/finally 全景
- └─ 9.2 抛异常 —— raise、raise from、自定义异常
- └─ 9.3 GIL 真相 —— 为什么有、影响了什么
- └─ 9.4 threading —— 线程的创建与生命周期
- └─ 9.5 锁与竞态 —— Lock、死锁、临界区
- └─ 9.6 queue —— 线程安全的生产者-消费者
- └─ 9.7 multiprocessing —— 绕开 GIL 的并行
- └─ 9.8 asyncio —— 事件循环与 async/await

§9.1 异常体系:try/except/else/finally 全景

异常是类,按继承树组织: `BaseException` 是根, `Exception` 是"可捕获的业务与系统错误"基类,再往下是 `ValueError`、`TypeError`、`KeyError`、`OSError` 等。捕获是按继承关系匹配的,所以捕获顺序要先子类后父类:

```
try:
    data = json.loads(raw)
    score = data["score"] / data["count"]
except KeyError:
    print("字段缺失")
except (TypeError, ValueError) as e:
    print(f"数据不合法: {e}")          # 多个异常共用一个分支
except ZeroDivisionError:
    print("count 为 0")
except Exception as e:
    print(f"未知错误: {e}")            # 兜底,但别滥用
else:
    print(f"无异常,正常处理 score={score}")
finally:
    print("无论如何都会执行:清理资源")
```

C(返回码 + errno)	Python(异常)
漏查返回值,错误被静默吞掉	不捕获异常就向上传播,不会被静默
错误路径与正常路径混在代码里	try 块聚焦正常逻辑,错误处理分离
errno 是全局变量,嵌套调用互相覆盖	异常对象携带完整上下文,链式传递
释放资源要手动检查每个分支	finally/with 保证清理必然执行

两个关键语义:else 分支"只有 try 无异常才执行"——把"预期成功后的动作"与"错误处理"分开;finally 无论 return 与否都执行,是资源清理的最后防线。注意 `except Exception` 捕获不了 `KeyboardInterrupt` 与 `SystemExit`(它们继承 `BaseException`),这是有意的设计。

误区:裸 `except:` 吞掉一切。 `except:` 连 `KeyboardInterrupt`、`SystemExit` 一起吞,程序 `Ctrl+C` 都退不出来;再配上空 `pass`,错误信息全部消失,排障靠猜。生产规范:捕获具体异常,处理不了的用 `raise` 原样重抛,或转成带上下文的业务异常。

§9.2 抛异常与自定义异常

主动抛异常是"前置校验"的 Python 方式: `raise ValueError(...)` 在入口处拦截非法参数,比让错误在深处爆发更容易定位。生产项目通常自定义异常基类,让调用方能按业务维度捕获:

```
class PaymentError(Exception):
    """支付业务异常基类:所有支付错误都继承它"""

class InsufficientBalanceError(PaymentError):
    """余额不足"""

class AccountLockedError(PaymentError):
    """账户被锁定"""

def pay(user: dict, amount: float) -> None:
    if user.get("locked"):
        raise AccountLockedError(f"账户 {user['name']} 已锁定")
    if user["balance"] < amount:
        raise InsufficientBalanceError(
            f"余额 {user['balance']} 不足 {amount}")
    user["balance"] -= amount

try:
    pay({"name": "李雷", "balance": 10, "locked": False}, 100)
except PaymentError as e:
    # 一个分支捕获整个业务族
    print(f"支付失败: {e}")
```

异常链 `raise ... from e` 保留"底层原因"——处理 `requests` 异常时尤其重要:

```

try:
    resp = requests.get(url, timeout=10)
    resp.raise_for_status()
    return resp.json()
except requests.Timeout as e:
    raise PaymentError(f"支付网关超时: {url}") from e
except requests.HTTPError as e:
    raise PaymentError(
        f"支付网关返回 {resp.status_code}") from e

```

输出里会显示完整的"直接原因"链,一眼看到最底层是连接超时还是解析失败。这是生产排障的核心工具,绝不写成裸 raise 丢上下文。

§9.3 GIL 真相:为什么有、影响了什么

GIL(全局解释器锁)是 CPython 的一个互斥锁:任何时刻只有一个线程在执行 Python 字节码。为什么存在?因为 CPython 的内存管理(引用计数)不是线程安全的,加一把全局锁让整个解释器原子化,代价是牺牲多核并行。它不影响"并发"——多个线程仍然轮流执行,只是不能同时推进。

C(pthread)	Python(threading)
多线程可同时跑在多核上,真并行	GIL 下字节码不并行,但 I/O 等待会让出锁
共享数据全靠自己加锁	同样要加锁;GIL 只保证字节码原子,不保证你代码原子
线程开销低,百万线程不现实	线程是系统线程,创建开销可观,数量控制在百级
计算密集任务靠线程加速	计算密集靠线程反而变慢(锁竞争),要用进程

GIL 对 I/O 密集任务几乎无感:网络请求、文件读写、数据库查询在等待期间会释放 GIL,其他线程照常运行。所以 web 服务、爬虫、API 调用用线程完全没问题;纯 CPU 计算(排序、图像处理)要并行,选 multiprocessing 或把热点交给 C 扩展。

误区:GIL 让我不需要加锁。 GIL 只保证单条字节码原子,不保证"读取-修改-写回"序列原子。count += 1 要经历取数、加一、写回三步,两个线程交错执行就会丢更新。共享可变数据的保护,Python 与 C 一样要靠锁,没有免费午餐。

§9.4 threading:线程的创建与生命周期

```
import threading
import time

def worker(name: str, delay: float) -> None:
    print(f"[{name}] 开始工作")
    time.sleep(delay)          # 模拟 I/O 等待
    print(f"[{name}] 完成")

threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(f"w{i}", 1.0))
    t.start()                  # 启动线程
    threads.append(t)

for t in threads:
    t.join()                   # 等待全部结束再退出
print("所有线程完成")
```

关键 API: `start()` 启动, `join()` 等待结束, `daemon=True` 的线程不阻塞进程退出(主线程结束即杀);线程间共享的是同一个进程内存,这正是竞态的温床。线程安全地收集结果,不要用全局 list 直接 `append`——用下面 §9.6 的 `queue`,或用 `concurrent.futures` 的线程池(生产首选)。线程函数的参数只能通过 `args` 元组传递,而且线程没有返回值通道——这正是 `queue` 与 `Future` 存在的意义,理解这一点,就理解了线程间通信为什么绕不开队列。

§9.5 锁与竞态:临界区、死锁

竞态(race condition)是并发程序的头号 bug:两个线程同时改一个变量,最终值取决于交错时机。标准解法是 **Lock**,把"读-改-写"包成临界区:

```
import threading

balance = 1000.0
lock = threading.Lock()

def transfer(amount: float) -> bool:
    """并发转账:先查后扣,临界区保护"""
    with lock:                  # 等价于 acquire/finally release
        if balance < amount:
            return False
        balance -= amount      # 读-判断-写 现在原子了
    return True
```

误区:锁的顺序不一致导致死锁。两个线程各持一把锁再等对方的锁,双方永远等下去——这就是死锁。防御规则:① 锁的获取顺序全局一致(如总是先锁 A 再锁 B);② 用 `with lock:` 而不是手动 `acquire/release`(异常时保证释放);③ 加锁范围最小化,不要在锁里做网络请求或 `sleep`;④ 实在复杂就用 `queue` 消除共享,无锁则无死锁。

另一个必懂概念:同一线程对同一把锁重复 with 会死锁——除非用 `threading.RLock` (可重入锁)。递归函数里加锁、锁里调用会再次加锁的同模块函数,都是 RLock 的典型场景。默认先写 Lock,出现重入需求再换。

§9.6 queue:线程安全的生产者-消费者

生产者-消费者是并发编程的基本架构:生产者产数据,消费者处理数据,中间用队列解耦。Python 的 `queue.Queue` 内部自带锁,是线程间传数据的标准容器——比"共享 list + 手写锁"少一半 bug:

```
import queue
import threading
import time

def producer(q: queue.Queue, n: int) -> None:
    for i in range(n):
        q.put(f"任务-{i}")          # 放任务,内部加锁
        time.sleep(0.05)

def consumer(q: queue.Queue, name: str) -> None:
    while True:
        item = q.get()              # 取任务,队列空则阻塞等待
        print(f"[{name}] 处理 {item}")
        q.task_done()              # 通知队列:一个任务完成

q = queue.Queue(maxsize=20)
threading.Thread(target=producer, args=(q, 10), daemon=True).start()
for i in range(2):
    threading.Thread(target=consumer, args=(q, f"c{i}"),
                      daemon=True).start()

q.join()                          # 等所有任务都被 task_done
print("全部消费完成")
```

三个方法组成了完整生命周期:put 放、get 取、task_done 报告完成,配套 join() 等待"所有已放任务完成"。队列满时 put 阻塞,队列空时 get 阻塞——生产里可以给超时(`q.get(timeout=5)`)或设置哨兵值(发 None 通知消费者退出)。

§9.7 multiprocessing:绕开 GIL 的并行

计算密集任务要真并行,用 `multiprocessing` 开多个进程——每个进程有自己的解释器与 GIL,多核真正同时计算。API 与 `threading` 高度相似,区别是内存不共享,数据要序列化后传递:

```
import multiprocessing as mp

def heavy(n: int) -> int:
    """模拟 CPU 密集:计算前 n 个数的平方和"""
    return sum(i * i for i in range(n))

if __name__ == "__main__":
    with mp.Pool(4) as pool:          # 4 个进程
        results = pool.map(heavy, [10_000_000] * 4)
    print(results)                    # 约 4 倍于单进程的耗时
```

C(fork/线程)	Python(进程选择)
I/O 密集:多线程	threading(受 GIL 影响小,轻量)
CPU 密集:多线程直接并行	multiprocessing.Pool(真并行,数据要可序列化)
共享内存传递结构体	进程间默认 pickle 传参,大对象开销高
进程启动代价可控	Python 进程启动重,池化复用是标配

注意三点:参数与返回值必须可 pickle(函数要模块级定义);Windows 上进程通过 spawn 创建,入口代码必须放 `if __name__ == "__main__":` 保护,否则无限递归;任务小而多时进程开销可能超过收益,先用 profile 数据说话。

§9.8 asyncio:事件循环与 async/await

asyncio 是单线程内的协作式并发:一个事件循环在多个协程之间切换,遇到 `await` 就切走,不阻塞也不占线程。适合海量 I/O 并发(万级连接)而线程做不到的场景:

```
import asyncio

async def fetch(name: str, delay: float) -> str:
    """协程:遇到 await 让出控制权,等待期间执行别的协程"""
    print(f"[{name}] 开始请求")
    await asyncio.sleep(delay)      # 模拟 I/O 等待(不占线程)
    print(f"[{name}] 响应到达")
    return name.upper()

async def main() -> None:
    tasks = [fetch("a", 1.0), fetch("b", 0.5), fetch("c", 0.2)]
    results = await asyncio.gather(*tasks)  # 并发跑三个协程
    print(results)                        # ['A', 'B', 'C']

if __name__ == "__main__":
    asyncio.run(main())                  # 创建事件循环并运行
```

理解要点:async def 定义协程;await 是让出点;asyncio.run 是入口;gather 并发收集结果。三个任务总耗时约 1 秒(最长者)而不是 1.7 秒——这就是并发的价值。写生产代码时,用 aiohttp 做 HTTP、asyncpg 连数据库,网络等待期间循环继续转。

误区:协程里混入阻塞调用。在 `async` 函数里直接 `requests.get` 或 `time.sleep`,会把整个事件循环卡死——所有协程一起等这一个阻塞调用。规则:`async` 世界里只能用异步库(`await` 版本);不得不用同步库时,丢到 `asyncio.to_thread` 里跑,别让它占住循环。

工程建议 1:并发选型口诀:I/O 密集且并发量适中(百级)用 `threading`;I/O 密集且海量(千级+)用 `asyncio`;CPU 密集用 `multiprocessing`;三者可以组合,但每层都要明确边界。

工程建议 2:所有并发代码都要回答三个问题:错误在哪捕获(线程里未捕获异常会静默消失,要包 `try/except` 并记录)、资源在哪释放(锁与连接用 `with`)、如何优雅停机(`join` 与 `sentinel` 值)。回答不清,上线必出事。

本章速查表

场景	写法
捕获异常	<code>except ValueError as e:</code> ,先子类后父类
多异常一支	<code>except (TypeError, ValueError):</code>
无异常才执行	<code>else:</code> 块
必然清理	<code>finally:</code> 或 <code>with</code>
抛出业务错误	<code>raise PaymentError("...")</code>
保留异常链	<code>raise X from e</code>
启动线程	<code>t = threading.Thread(target=f, args=()); t.start()</code>
等待线程	<code>t.join()</code>
临界区	<code>with lock:</code> 范围最小化
可重入锁	<code>threading.RLock()</code>
线程传数据	<code>queue.Queue</code> + <code>put/get/task_done</code>
CPU 并行	<code>mp.Pool(n).map(f, items)</code> ,须 <code>__main__</code> 保护
协程并发	<code>await asyncio.gather(*tasks)</code>
事件循环入口	<code>asyncio.run(main())</code>
<code>async</code> 里跑同步库	<code>await asyncio.to_thread(sync_fn, ...)</code>
线程内未捕获异常	不会打印!包 <code>try/except</code> 自己记录

最小必会清单

1. 能写出 `try/except/else/finally` 完整结构,并解释 `else` 与 `finally` 的语义差异
2. 能设计自定义异常层次(业务基类 + 具体子类),并用 `raise from` 保留根因
3. 能解释 GIL 是什么、为什么存在,以及它对 I/O 密集与 CPU 密集任务的不同影响
4. 能用 `with lock` 保护临界区,能识别死锁场景并说出防御规则
5. 能用 `queue.Queue` 实现生产者-消费者,解释 `put/get/task_done/join` 的作用
6. 能说出线程、进程、协程三种并发的适用场景,并写一个最小的 `async/await` 例子

自测题

Q

Q1. else 与 finally 的区别是什么?try 里 return 了,finally 还执行吗?

答案:else 只在 try 无异常时执行,是"成功路径"的延续;finally 无条件执行,用于清理。try 里 return 时,finally 先执行再返回——包括 return 表达式求值完成后、真正退出函数前。若 finally 里也有 return,它会覆盖 try 的返回值(生产上避免在 finally 写 return)。

Q

Q2. except: 裸捕获有什么危害?什么情况下用它合理?

答案:它连 KeyboardInterrupt、SystemExit、GeneratorExit 一起吞,程序无法被 Ctrl+C 停止;空 pass 更让错误彻底消失。合理场景:最外层"记录日志并优雅退出"的兜底,且必须记录完整异常(traceback.format_exc());除此之外一律捕获具体异常类型。

Q

Q3. 两个线程同时执行 count += 1 一万次,count 为什么不是 20000?GIL 为什么没保护它?

答案:count += 1 是"取数、加一、写回"三步字节码,GIL 只保证单条字节码不被其他线程插入,不保证这三步连续。线程 A 取到 100 后让出锁,线程 B 也取到 100 并写回 101,A 再写回 101——丢了一次更新。保护办法:with lock: 包住,或用 queue 转移所有权。

Q

Q4. 死锁的四个必要条件是什么?代码层面如何预防?

答案:互斥、持有并等待、不可抢占、循环等待。预防:① 全局一致的锁顺序(打破循环等待);② with lock 自动释放(不可抢占下的异常安全);③ 锁范围最小化,锁内不做 I/O;④ 用 queue 等结构消除共享,无共享则无锁;⑤ 复杂场景用锁超时(acquire(timeout=))并记录告警。

Q

Q5. 一个爬虫要并发抓 10000 个 URL,线程、进程、协程各有什么利弊?你会选哪个?

答案:线程:百级可行,万级线程创建与切换开销过大;进程:能真并行但进程数多时资源重、传参序列化开销大;协程:单线程内万级并发毫无压力,I/O 等待自动切换,是首选。选 asyncio + aiohttp,连接池控制并发上限(semaphore),超时与重试照章执行。

Q

Q6. 线程函数里抛了异常,主线程为什么没看到?怎么处理?

答案:线程异常只属于该线程,解释器不会把它传回主线程;线程静默死亡,主线程 join 返回,一切"正常"。处理:线程函数内 try/except 记录日志;需要结果的场景用 concurrent.futures.Future——future.result() 会把异常重新抛给调用方,异常与结果同路返回。

章末实验题:生产者-消费者任务流水线

实验 9.1 日志处理流水线(覆盖:\$9.3-\$9.6)

需求:① 生产者线程生成 200 条模拟日志(随机级别与耗时),放入有界队列;② 三个消费者线程从队列取日志,按级别统计到共享计数器(必须加锁),并模拟处理耗时;③ 队列满时 put 阻塞、空时 get 阻塞,用 task_done/join 保证"全部处理完才结束";④ 增加一个"异常注入"开关:某条日志处理时抛业务异常,消费者捕获并记录到错误队列,最后汇总打印;⑤ main 汇总输出各级别统计与错误数,验证并发正确性(加锁与不加锁的结果对比)。

覆盖知识点:GIL 与线程(\$9.3、\$9.4)/锁与临界区(\$9.5)/queue 生产消费(\$9.6)/异常处理(\$9.1、\$9.2)。

实验环境:Python 3.10+,单文件 `pipeline.py`。

```

# pipeline.py — 日志处理流水线(覆盖第9章全部知识点)
import queue
import random
import threading
import time

LEVELS = ("INFO", "WARN", "ERROR")
STATS_LOCKED = {lv: 0 for lv in LEVELS} # 加锁版本
STATS_RAW = {lv: 0 for lv in LEVELS} # 不加锁对照
stats_lock = threading.Lock()
error_q = queue.Queue() # 错误汇报通道
FAIL_EVERY = 17 # 每 17 条注入一次异常

def make_log(i: int) -> dict:
    """模拟一条日志:级别随机,耗时随机"""
    return {"id": i,
            "level": random.choice(LEVELS),
            "cost": random.uniform(0.005, 0.02)}

def producer(q: queue.Queue, total: int) -> None:
    """生产者:生成日志放入有界队列,放不下就阻塞"""
    for i in range(total):
        q.put(make_log(i)) # 队列满时自动阻塞
    q.put(None) # 哨兵:通知消费者退出

def consumer(q: queue.Queue, name: str, use_lock: bool) -> None:
    """消费者:取日志、统计、模拟处理;遇哨兵退出"""
    while True:
        item = q.get()
        if item is None: # 哨兵值,退出循环
            q.task_done()
            break
        try:
            if item["id"] % FAIL_EVERY == 0:
                raise ValueError(f"日志 {item['id']} 内容损坏")
            time.sleep(item["cost"]) # 模拟处理耗时
        except ValueError as e:
            error_q.put(str(e))
            q.task_done()
            continue
        if use_lock:
            with stats_lock: # 临界区:读-改-写原子化
                STATS_LOCKED[item["level"]] += 1
        else:
            STATS_RAW[item["level"]] += 1 # 竞态对照组
        q.task_done()

```

```

def run(use_lock: bool) -> tuple:
    """跑一轮:总耗时、统计结果、错误数"""
    q = queue.Queue(maxsize=30)          # 有界队列
    error_q.queue.clear()
    threads = []
    threads.append(threading.Thread(
        target=producer, args=(q, 200), daemon=True))
    for i in range(3):                  # 三个消费者
        threads.append(threading.Thread(
            target=consumer, args=(q, f"c{i}", use_lock),
            daemon=True))
    for t in threads:
        t.start()
    q.join()                            # 等所有任务完成
    for t in threads:
        t.join()
    stats = dict(STATS_LOCKED if use_lock else STATS_RAW)
    return stats, error_q.qsize()

def main() -> None:
    random.seed(42)
    locked_stats, locked_err = run(True)
    print("加锁统计:", locked_stats, "错误:", locked_err)
    raw_stats, _ = run(False)
    print("不加锁统计:", raw_stats)
    print("合计(应为 200 - 错误数):",
          sum(raw_stats.values()))

if __name__ == "__main__":
    main()

```

设计说明:① 有界队列 maxsize=30 让 put 阻塞可被观察,哨兵 None 让消费者干净退出;② 加锁与不加锁两轮对照,能直接看到统计丢失(两轮合计明显小于 200 减错误数);③ 异常注入走独立的错误队列,不污染主数据通道——这正是“错误与数据分离”的生产姿势;④ 每轮用独立 queue 与线程组,避免状态串扰;⑤ main 里 seed 固定,结果可复现,适合自动化验证。

下一章预告:第10章 类型标注与元编程——TypeVar 与 Generic、Protocol 结构化协议、元类与 __init_subclass__、描述符协议。让动态语言在大型项目里获得静态检查的护航。